

Kernax

User's Guide

Ziad Iziz

INFOB318 — UNamur

2026

Table des matières

1	Introduction	2
1.1	Qu'est-ce que Kernax ?	2
1.2	Pour qui ?	2
1.3	Ce que Kernax ne fait pas	2
2	Installation	3
2.1	Prérequis	3
2.2	Procédure	3
3	Exemples d'utilisation	4
3.1	Utiliser un Processus Gaussien	4
3.2	Utiliser un SVM	4
3.3	Utiliser la KRR	4
4	Erreurs fréquentes	5
4.1	X must be 2D	5
4.2	The noise can't be negative	5
4.3	Divergence de la loss	5

Chapitre 1

Introduction

1.1 Qu'est-ce que Kernax ?

Kernax est une bibliothèque Python d'optimisation de kernels développée par Simon Lejoly. Elle a été optimisée pour tirer pleinement partie de la compilation XLA de jax.

1.2 Pour qui ?

Kernax s'adresse aux utilisateurs recherchant une librairie de kernels rapide et exhaustive.

1.3 Ce que Kernax ne fait pas

Kernax ne prend pas en charge nativement la classification multi classes pour les SVM, entre autres. Kernax n'est aussi pas une librairie de machine learning fournissant des likelihood functions etc..

Chapitre 2

Installation

2.1 Prérequis

Kernax nécessite Python 3.10 ou plus, les dépendances principales sont Optax et JAX. L'ensemble des versions requises est listé dans requirements.txt

2.2 Procédure

La librairie peut se pip install : `pip install kernax-ml`

Chapitre 3

Exemples d'utilisation

Cette section présente les modèles à travers des scénarios concrets, les différents notebooks illustrent ces exemples de manière exhaustive.

3.1 Utiliser un Processus Gaussien

Tout d'abord on choisit un Kernel, par exemple SE avec une length scale de 1.0. Puis on crée un objet GP en lui passant ce kernel et un niveau de noise, l'appel à fit optimise automatiquement les hyperparamètres du kernel par maximisation du log vraisemblance marginal. Une fois le modèle entraîné, predict renvoie la moyenne postérieure. C'est le modèle le plus complet des exemples.

3.2 Utiliser un SVM

Notre SVM va fonctionner en 2 modes, selon si un kernel est fourni ou non au début. Sans kernel, on opère dans un espace primal, la fit function va minimiser la hinge loss, pour trouver le poids w et le biais b . La fonction predict renvoie les classes prédites (+1 ou -1) selon la fonction $\hat{y} = \text{sign}(w^T x + b)$. Avec Kernel le modèle bascule automatiquement en formulation duale, et utilise les multiplicateurs de Lagrange. Les exemples sont disponibles dans SVMExamples.ipynb.

3.3 Utiliser la KRR

La KRR a la particularité d'avoir ses données d'entraînements passées au constructeur et non à fit. On instancie d'abord KRR(Params) puis on appelle Fit avec seulement les cibles. Le modèle résout $(K + \lambda I)\alpha = y$ par décomposition de Cholesky pour trouver les coefficients duaux et predict retourne les valeurs prédites.

Chapitre 4

Erreurs fréquentes

4.1 X must be 2D

Les données d'entraînements doivent toujours être passé sous la forme de tableau 2D de dimensions (number of samples, number of features). Les vecteurs 1D doivent aussi être reshape.

4.2 The noise can't be negative

Le noise doit aussi être strictement positif

4.3 Divergence de la loss